计算机网络技术

本节内容

封装成帧

错误检测

数据链路层（第二层）

数据链路层的核心任务是在物理层提供的比特流基础上，提供可靠的数据传输，并确保数据能正确地从一个设备发送到另一个设备

向网络层提供一个定义良好的服务接口

将数据报封装成帧 / 从比特流中解析帧

错误检测和纠正

调节数据流

介质访问控制

帧（Frame）

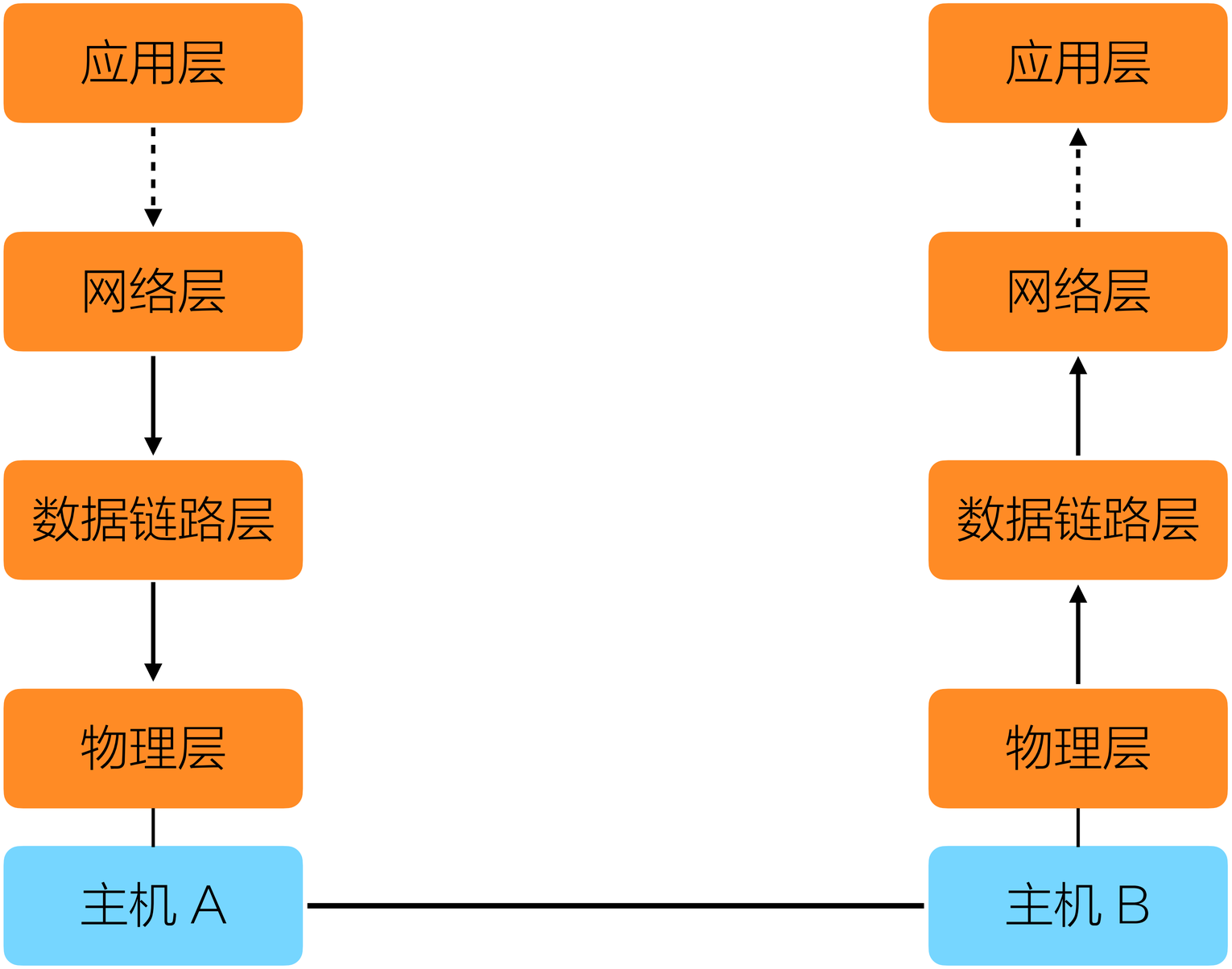
在数据链路层，数据以“帧（Frame）”的形式进行传输，每一帧通常包含以下三部分：

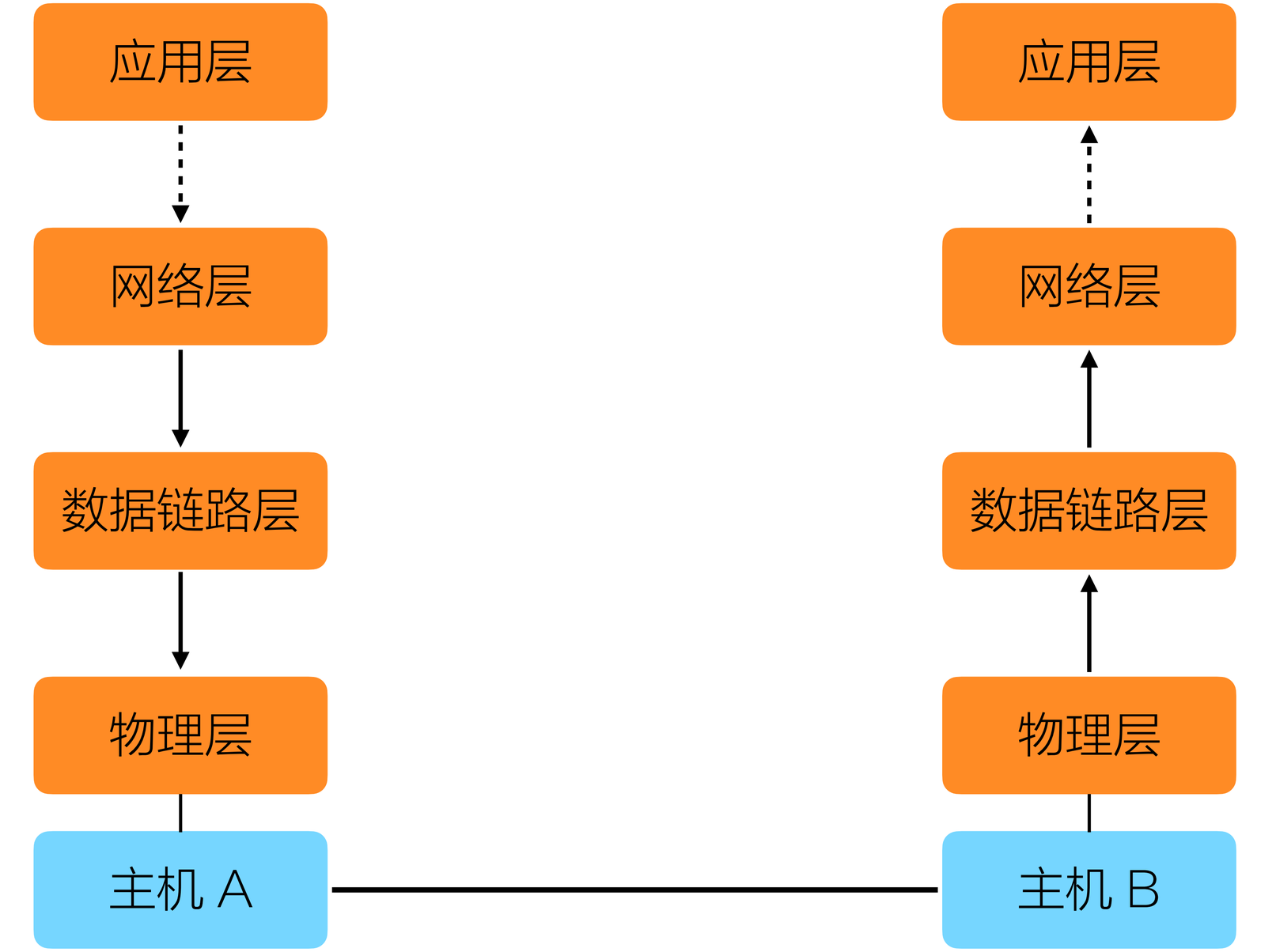
帧头（Header）：包含地址信息（源地址、目的地址）、帧类型、控制信息等。

数据（payload）：承载来自网络层的数据报文（通常是IP数据报）

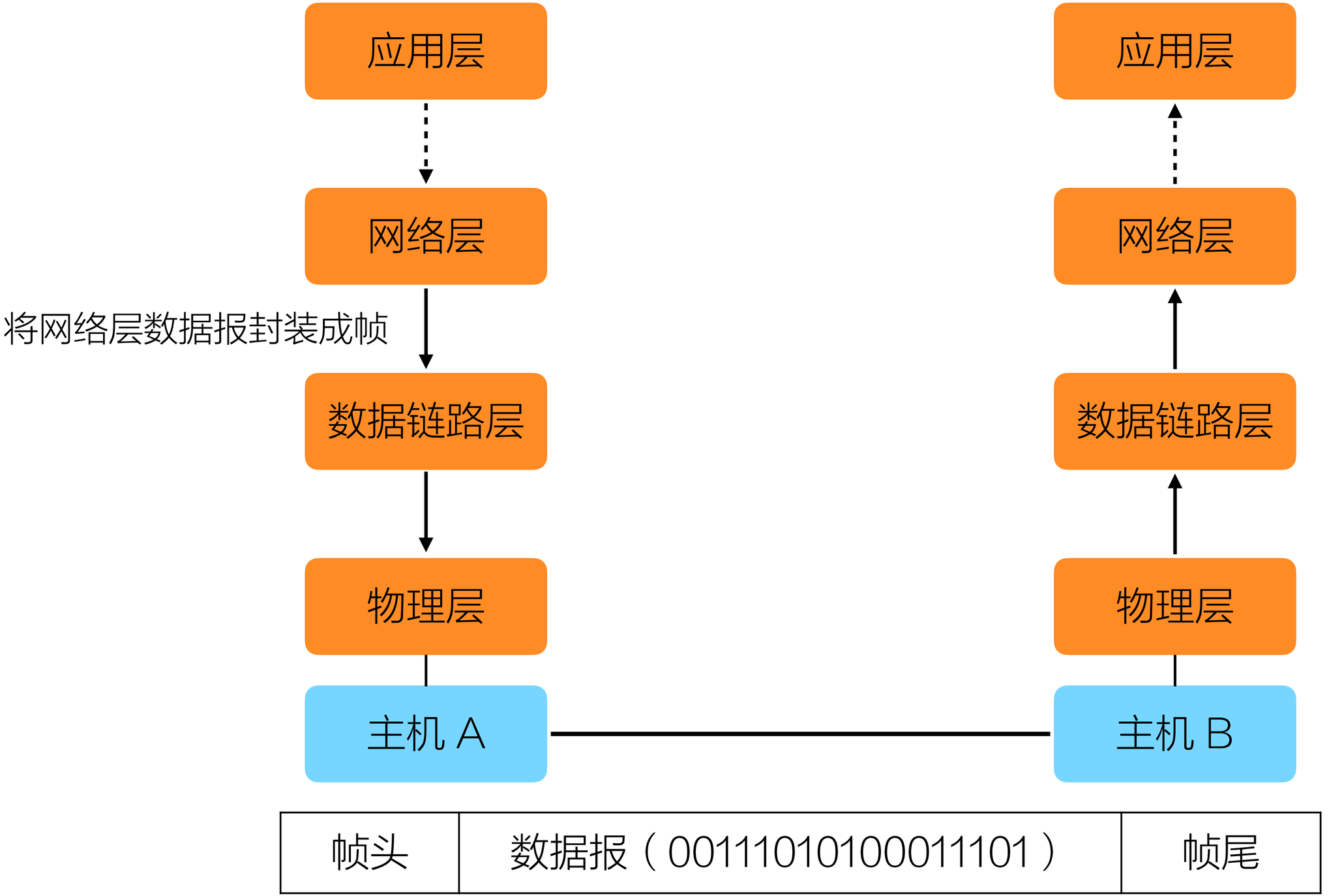
帧尾（Trailer）：通常包含校验和用于检测数据传输错误

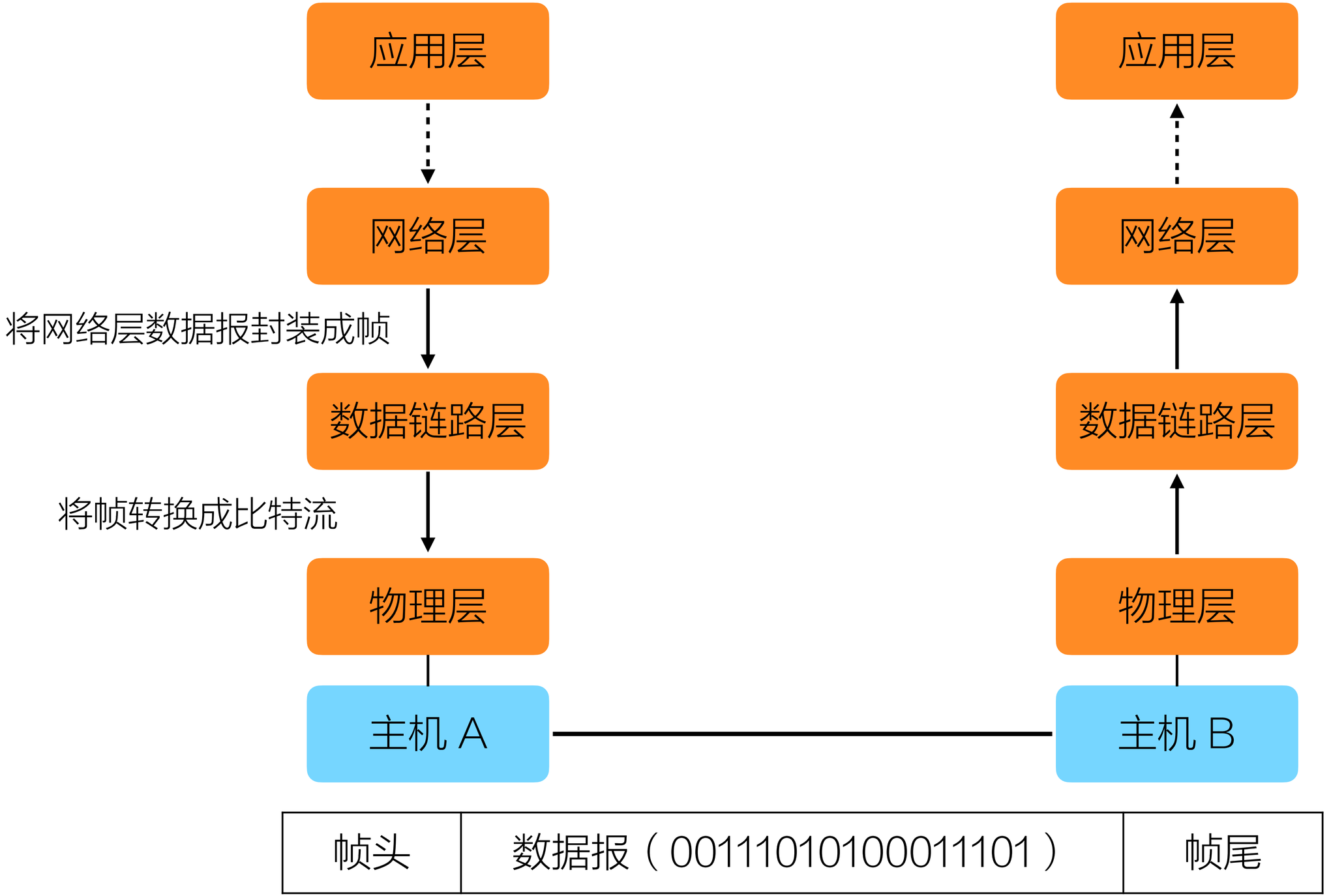
帧头	数据报（0011101010001101）	帧尾

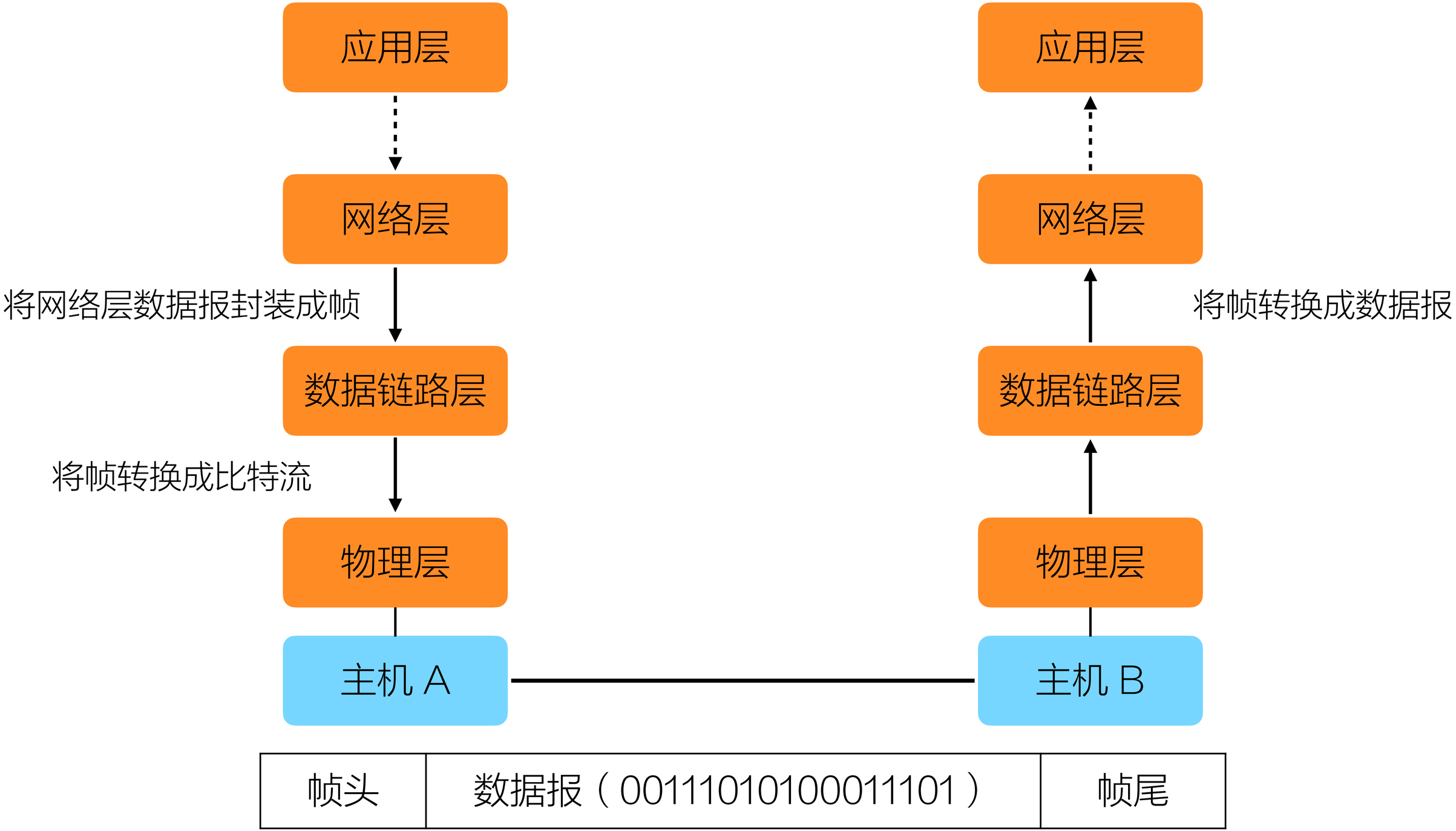

应用层
网络层
数据链路层
物理层
主机 A
应用层
网络层
数据链路层
物理层
主机 B

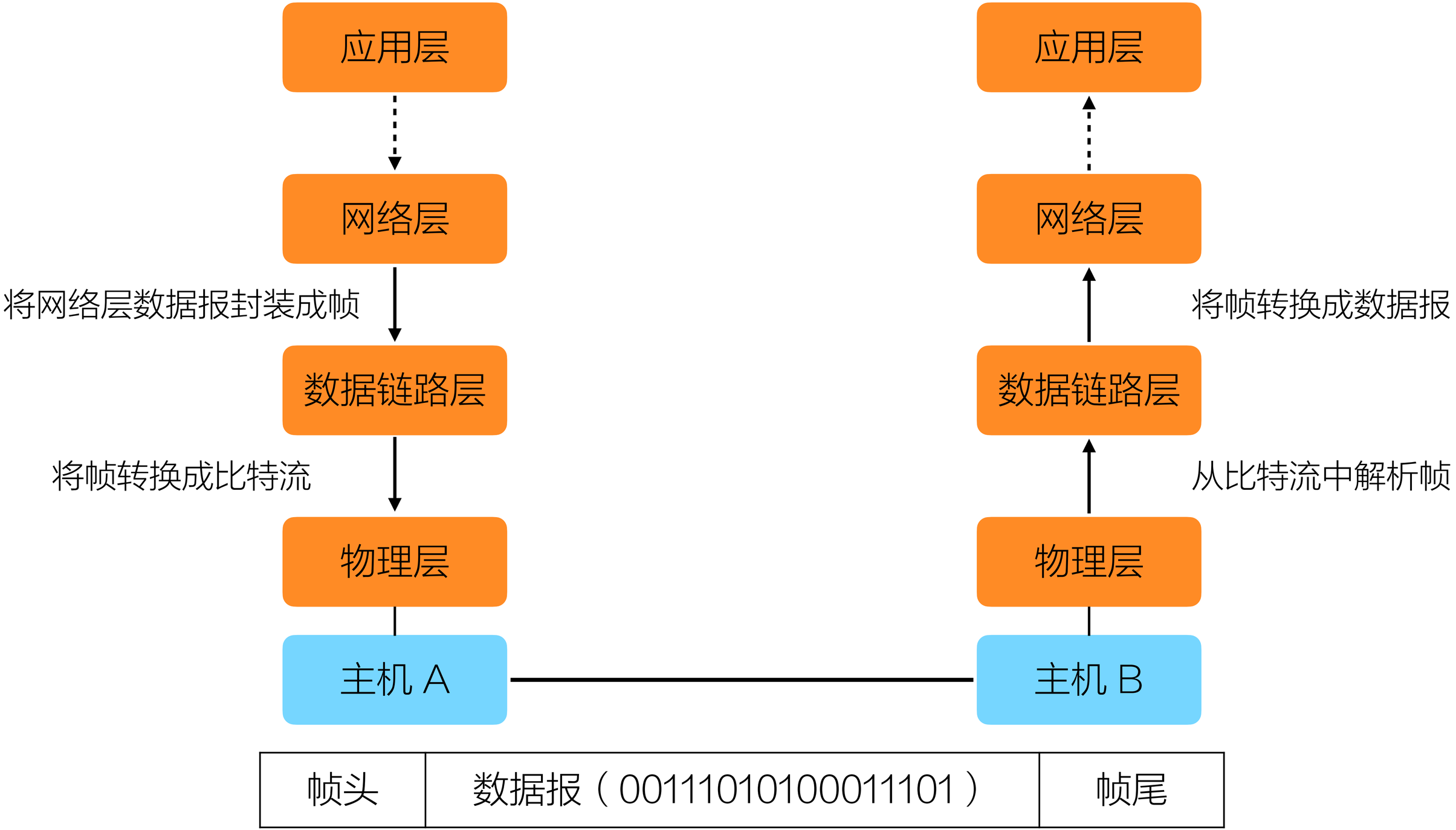


帧头	数据报（0011101010100011101）	帧尾


应用层
网络层
将网络层数据报封装成帧
数据链路层
物理层
主机 A
应用层
网络层
数据链路层
物理层
主机 B
帧头
数据报（0011101010000111101）
帧尾

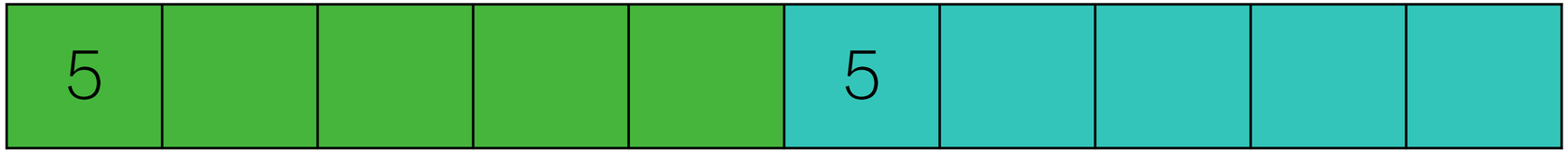

应用层
网络层
将网络层数据报封装成帧
数据链路层
将帧转换成比特流
物理层
主机 A
应用层
网络层
数据链路层
物理层
主机 B
帧头
数据报（0011101010000111101）
帧尾


应用层
网络层
将网络层数据报封装成帧
数据链路层
将帧转换成比特流
物理层
主机 A
应用层
网络层
将帧转换成数据报
数据链路层
物理层
主机 B
帧头
数据报（0011101010000111101）
帧尾


应用层
网络层
将网络层数据报封装成帧
数据链路层
将帧转换成比特流
物理层
主机 A
应用层
网络层
将帧转换成数据报
数据链路层
从比特流中解析帧
物理层
主机 B
帧头
数据报（0011101010000111101）
帧尾

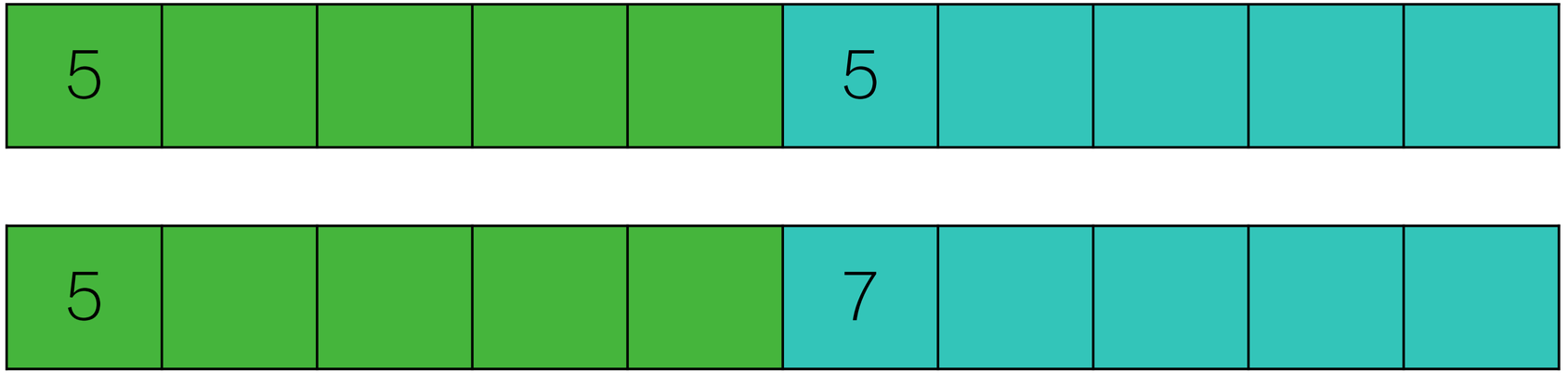
封装成帧 – 字节计数法

在帧的头部添加一个字段，指明该帧的字节数。



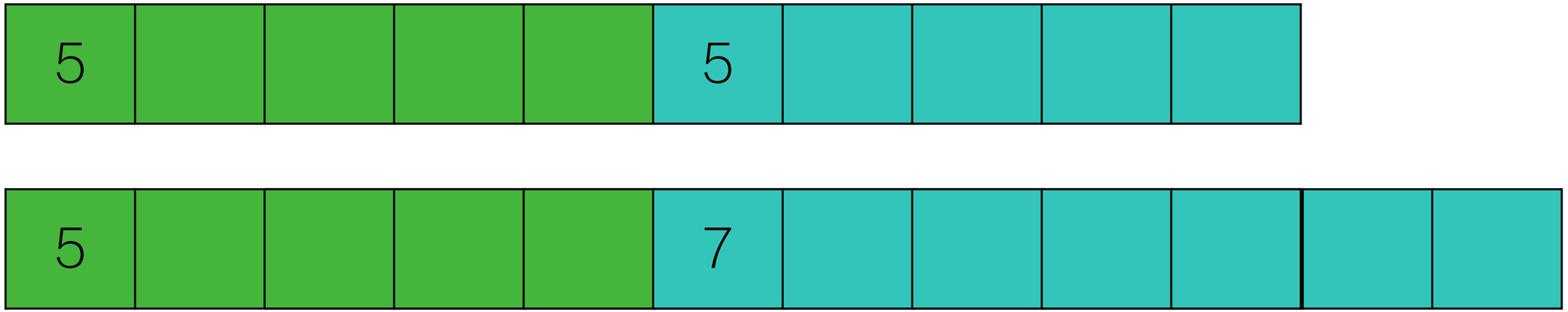
封装成帧 – 字节计数法

在帧的头部添加一个字段，指明该帧的字节数。



封装成帧 – 字节计数法

在帧的头部添加一个字段，指明该帧的字节数。



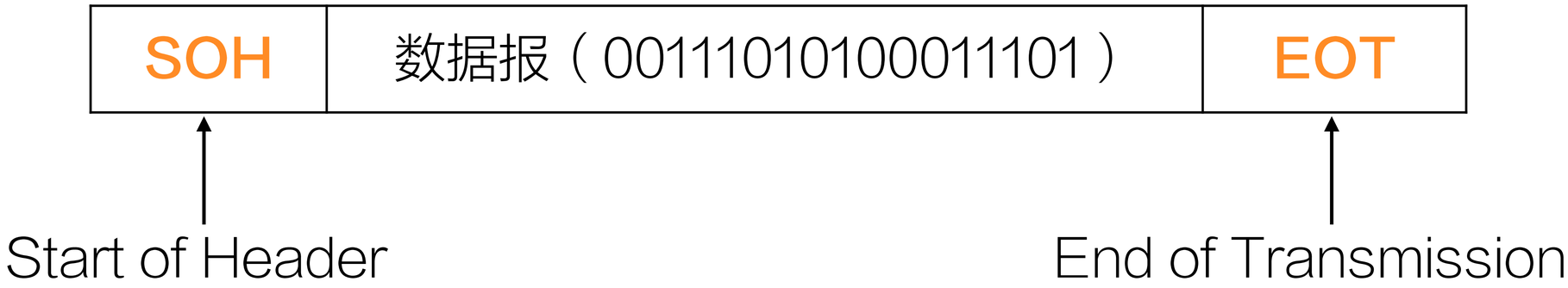
封装成帧 – 字节计数法

优点：简单易实现，占用的额外开销少

缺点：如果计数字段的数据**被传输错误**，会导致**数据同步错误**，接收端可能会错误地解析数据

封装成帧 – 字符填充法

使用特殊字符（如 SOH 代表帧起始，EOT 代表帧结束），并在数据部分中避免特殊字符的冲突。



ASCII 值	控制字符	ASCII 值	控制字符	ASCII 值	控制字符	ASCII 值	控制字符
0	NUT	32	(space)	64	@	96	、
1	SOH	33	!	65	A	97	a
2	STX	34	″	66	B	98	b
3	ETX	35	#	67	C	99	c
4	EOT	36	$	68	D	100	d
5	ENQ	37	%	69	E	101	e
6	ACK	38	&	70	F	102	f
7	BEL	39	`	71	G	103	g
8	BS	40	(	72	H	104	h
9	HT	41	)	73	I	105	i
10	LF	42	*	74	J	106	j
11	VT	43	+	75	K	107	k
12	FF	44	,	76	L	108	l
13	CR	45	—	77	M	109	m
14	SO	46	.	78	N	110	n
15	SI	47	/	79	O	111	o
16	DLE	48	0	80	P	112	p
17	DCI	49	1	81	Q	113	q
18	DC2	50	2	82	R	114	r
19	DC3	51	3	83	X	115	s
20	DC4	52	4	84	T	116	t
21	NAK	53	5	85	U	117	u
22	SYN	54	6	86	V	118	v
23	TB	55	7	87	W	119	w
24	CAN	56	8	88	X	120	x
25	EM	57	9	89	Y	121	y
26	SUB	58	:	90	Z	122	z
27	ESC	59	;	91	[	123	{
28	FS	60	<	92	\	124	|
29	GS	61	=	93	]	125	}
30	RS	62	>	94	^	126	~

ASCII 值	控制字符	ASCII 值	控制字符	ASCII 值	控制字符	ASCII 值	控制字符
0	NUT	32	（space)	64	@	96	、
1	SOH	33	!	65	A	97	a
2	STX	34	″	66	B	98	b
3	ETX	35	#	67	C	99	c
4	EOT	36	$	68	D	100	d
5	ENQ	37	%	69	E	101	e
6	ACK	38	&	70	F	102	f
7	BEL	39	`	71	G	103	g
8	BS	40	(	72	H	104	h
9	HT	41	)	73	I	105	i
10	LF	42	*	74	J	106	j
11	VT	43	+	75	K	107	k

封装成帧 – 字符填充法

SOH	数据报（0011110101000111101）	EOT

封装成帧 – 字符填充法



封装成帧 – 字符填充法

如果数据中包含 SOH 或 EOT，则用 ESC（转义字符）来填充，避免误判。



封装成帧 – 字符填充法

如果数据中包含 SOH 或 EOT，则用 ESC（转义字符）来填充，避免误判。



封装成帧 – 字符填充法

如果数据中包含 SOH 或 EOT，则用 ESC（转义字符）来填充，避免误判。



封装成帧 – 字符填充法

优点：解决了字节计数法的同步问题，适用于字符流

缺点：增加了额外的转义字符，可能会占用带宽

封装成帧 – 比特填充法

使用特定的比特模式作为帧的起始和终止标志，并在数据部分中避免比特模式的冲突。

01111110	01111110111111101	**01111110**

封装成帧 – 比特填充法

使用特定的比特模式作为帧的起始和终止标志，并在数据部分中避免比特模式的冲突。

数据中如果出现连续 5 个 1，则自动填充 0。

01111110	01111110111111101	**01111110**

封装成帧 – 比特填充法

使用特定的比特模式作为帧的起始和终止标志，并在数据部分中避免比特模式的冲突。

数据中如果出现连续 5 个 1，则自动填充 0。

01111110	01111101011111011101	01111110

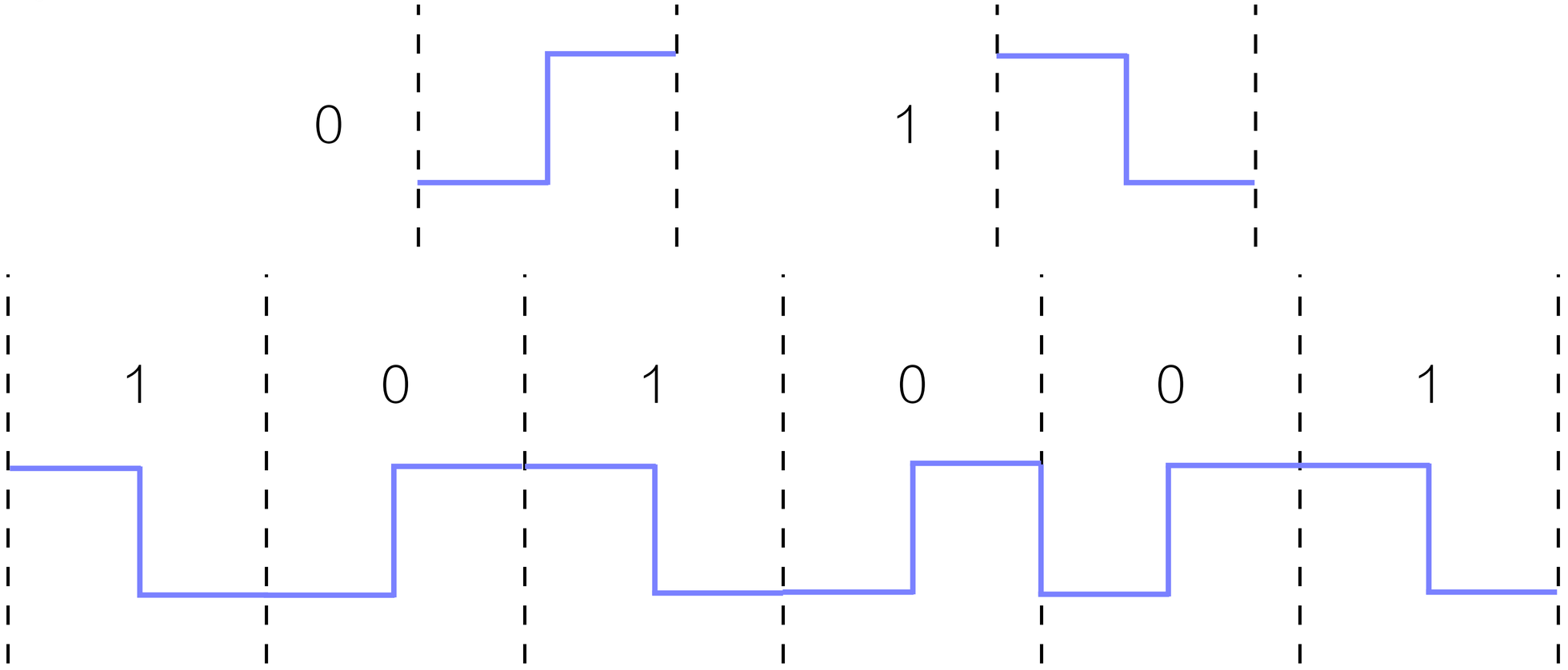
封装成帧 – 比特填充法

优点：适用于任何比特流，不局限于字符编码（比字符填充更通用）

缺点：需要额外的比特填充，稍微增加了传输开销

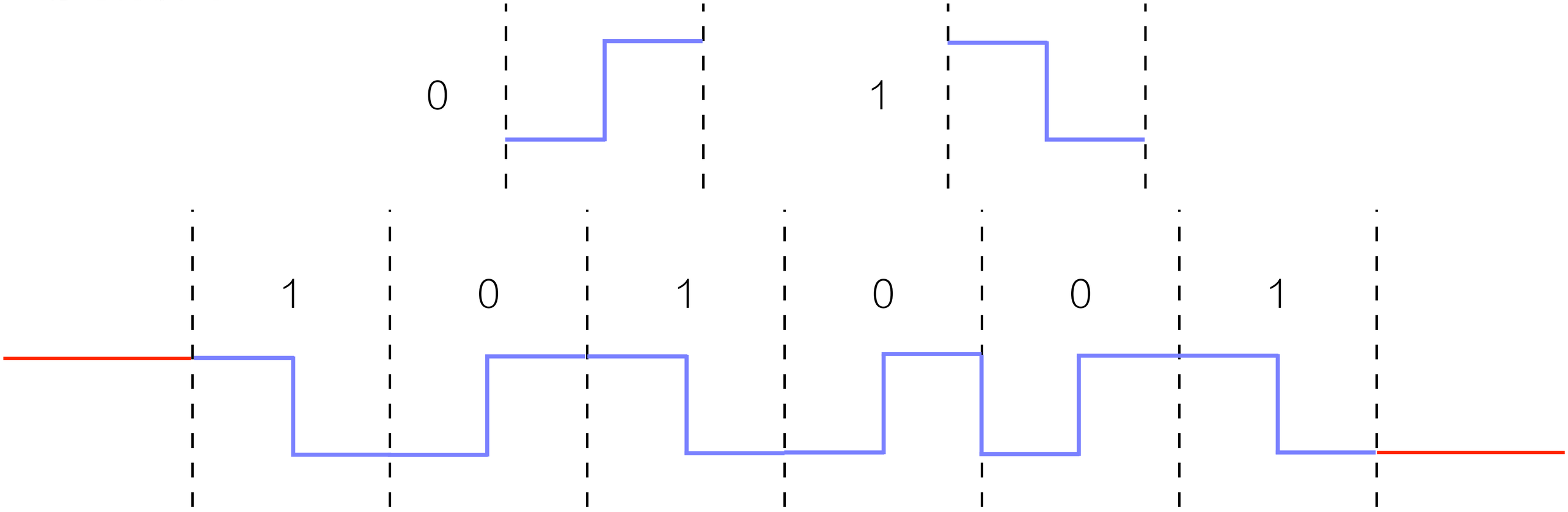
封装成帧 – 物理层编码法

数据报封装成帧后，直接在物理层使用信号的变化来标识帧的起始和结束，而不是依赖特定的比特模式。



封装成帧 – 物理层编码法

数据报封装成帧后，直接在物理层使用信号的变化来标识帧的起始和结束，而不是依赖特定的比特模式。



封装成帧 – 物理层编码法

优点：不需要填充比特，不会增加额外的开销

缺点：依赖物理层的编码方式，不适用于所有链路层协议

错误检测

数据在传输过程中，由于各种因素（如干扰、噪声、信号衰减），可能导致比特错误（比特翻转）。数据链路层提供错误检测机制，保证数据传输的可靠性，使得接收端能检测出数据的错误并采取相应措施（如请求重传）。

错误检测 – 常见的检测方法

奇偶校验（Parity Check）

检验和（Checksum）

循环冗余校验（CRC）

错误检测 – 奇偶校验（Parity Check）

奇校验还是偶校验是发送方与接收方事先协商好的

不论是奇校验还是偶校验，判断依据都是数据中 1 的个数的奇偶性

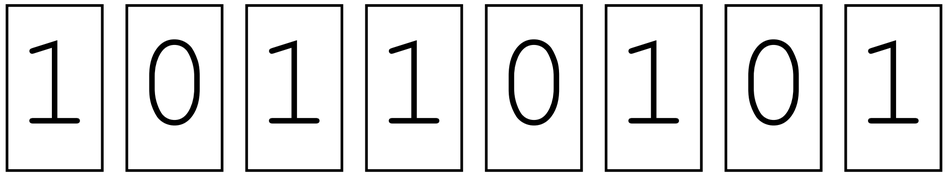
数据的 1 的个数 + 校验位 = 满足奇校验或偶校验的要求

错误检测 – 奇偶校验（Parity Check）

奇校验还是偶校验是发送方与接收方事先协商好的

不论是奇校验还是偶校验，判断依据都是数据中 1 的个数的奇偶性

数据的 1 的个数 + 校验位 = 满足奇校验或偶校验的要求

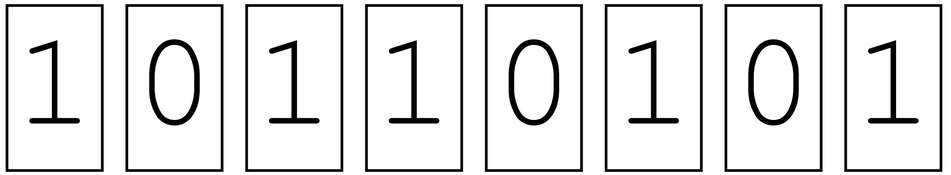


错误检测 – 奇偶校验（Parity Check）

奇校验还是偶校验是发送方与接收方事先协商好的

不论是奇校验还是偶校验，判断依据都是数据中 1 的个数的奇偶性

数据的 1 的个数 + 校验位 = 满足奇校验或偶校验的要求



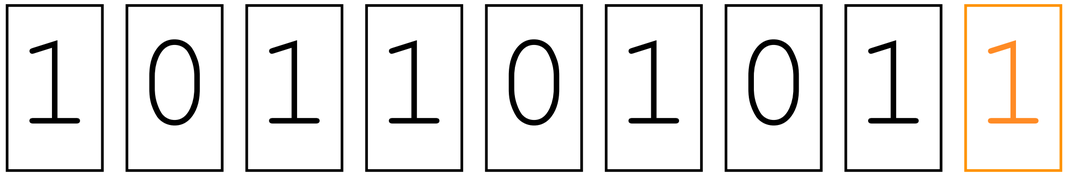
偶校验?

错误检测 – 奇偶校验（Parity Check）

奇校验还是偶校验是发送方与接收方事先协商好的

不论是奇校验还是偶校验，判断依据都是数据中 1 的个数的奇偶性

数据的 1 的个数 + 校验位 = 满足奇校验或偶校验的要求



偶校验?

错误检测 – 奇偶校验（Parity Check）

奇校验还是偶校验是发送方与接收方事先协商好的

不论是奇校验还是偶校验，判断依据都是数据中 1 的个数的奇偶性

数据的 1 的个数 + 校验位 = 满足奇校验或偶校验的要求

1	0	1	1	0	1	0	1

奇校验?

错误检测 – 奇偶校验（Parity Check）

奇校验还是偶校验是发送方与接收方事先协商好的

不论是奇校验还是偶校验，判断依据都是数据中 1 的个数的奇偶性

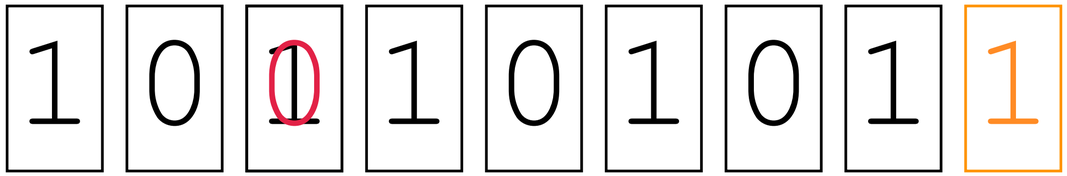
数据的 1 的个数 + 校验位 = 满足奇校验或偶校验的要求

1	0	1	1	0	1	0	1	0

奇校验?

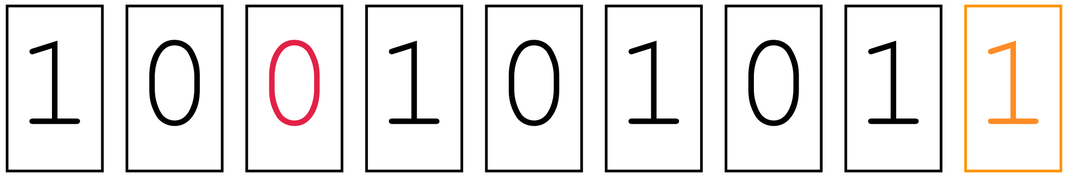
错误检测 – 奇偶校验（Parity Check）

采用偶校验



错误检测 – 奇偶校验（ Parity Check ）

采用偶校验



错误检测 – 奇偶校验（Parity Check）

优点：实现简单，开销小

缺点：检测能力弱，只能检测奇数个比特的错误，偶数个比特错误无法检测

错误检测 – 校验和（Checksum）

校验和是一种常用的错误检测方法，广泛用于TCP/IP协议中，具体操作方法如下：

- 发送方将待发送的数据分成若干段（通常16位为一组）
- 将这些数据段进行二进制求和
- 若求和过程中产生进位，则将进位部分回卷相加（循环进位）
- 最后将求和结果取反，作为校验和附加在数据后发送
- 接收方验证后如果结果全部是 1 说明数无误，否则数据有问题

错误检测 – 校验和（Checksum）

假设发送端数据如下

数据段1：0110011001100000

数据段2：0101010101010101

错误检测 – 校验和（Checksum）

计算校验和

```
  0110011001100000
+ 0101010101010101
------------------
```

错误检测 – 校验和（Checksum）

计算校验和

$$
\begin{array}{rr}
 & 0110011001100000 \\
+ & 0101010101010101 \\
\hline
 & 1011101110110101
\end{array}
$$

错误检测 – 校验和（Checksum）

将求和结果取反

1011101111010101

错误检测 – 校验和（Checksum）

将求和结果取反

1011101110110101

0100010001001010

错误检测 – 校验和（Checksum）

发送数据 + 校验和

数据段 1：0110011001100000

数据段 2：0101010101010101

校验和：0100010001001010

错误检测 – 校验和（Checksum）

接收端验证数据的正确性

错误检测 – 校验和（Checksum）

接收端验证数据的正确性

```
  0110011001100000   数据段 1
+ 0101010101010101   数据段 2
------------------
  1011101110110101
```

错误检测 – 校验和（Checksum）

接收端验证数据的正确性

```
  0110011001100000   数据段 1
+ 0101010101010101   数据段 2
------------------
  1011101110110101
+ 0100010001001010   校验和
------------------
```

错误检测 – 校验和（Checksum）

接收端验证数据的正确性

```
  0110011001100000   数据段 1
+ 0101010101010101   数据段 2
------------------
  1011101110110101
+ 0100010001001010   校验和
------------------
  1111111111111111
```

错误检测 – 校验和（Checksum）

带进位的校验和计算

数据段1：1111111111111111

数据段2：0000000000000001

错误检测 – 校验和（Checksum）

带进位的校验和计算

```
  1111111111111111
+ 0000000000000001
------------------
```

错误检测 – 校验和（Checksum）

带进位的校验和计算

```
  1111111111111111
+ 0000000000000001
------------------
  1000000000000000
```

错误检测 – 校验和（Checksum）

带进位的校验和计算

$$
\begin{array}{r}
 \\
+ \\
\hline
1000000000000000000
\end{array}
$$

错误检测 – 校验和（Checksum）

带进位的校验和计算

$$
\begin{array}{r}
0000000000000000 \\
+\qquad\qquad\qquad 1 \\
\hline
0000000000000001
\end{array}
$$

错误检测 – 校验和（Checksum）

优点：比奇偶校验强，能检测更多错误，简单高效，计算量小

缺点：不能检测所有错误（例如数据中部分比特位置翻转后总和仍相同，无法发现）

错误检测 – 循环冗余校验（Cyclic Redundancy Check）

CRC 是一种广泛用于数据链路层的错误检测方法，常用于以太网、HDLC、PPP 等协议。

- 发送方和接收方事先约定一个叫作生成多项式的二进制序列
- 生成多项式如果有 n 位，发送方在发送数据末尾添加 n-1 个 0
- 发送方使用末尾添加 0 的数据除以生成多项式，得到的余数为 CRC 码
- 发送方发送原始数据 + CRC
- 接收端收到数据后，用同样的生成多项式去除。如果余数为0，则数据无误；否则数据有误。

错误检测 – 循环冗余校验（Cyclic Redundancy Check）

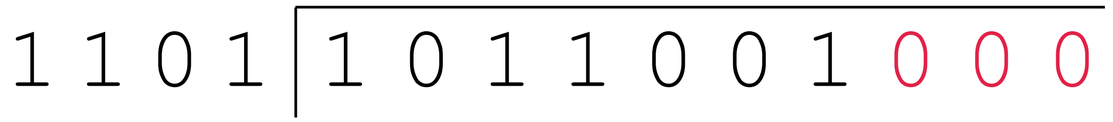
原始数据：1011001

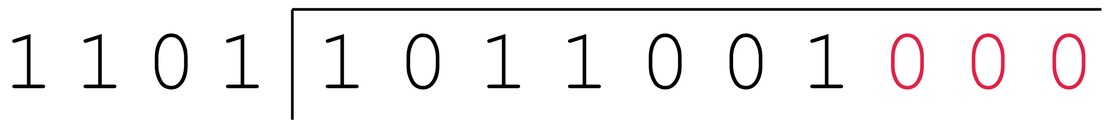
生成多项式：1101

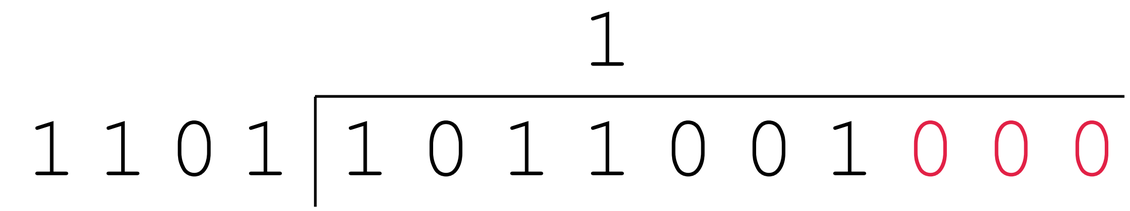
错误检测 – 循环冗余校验（Cyclic Redundancy Check）

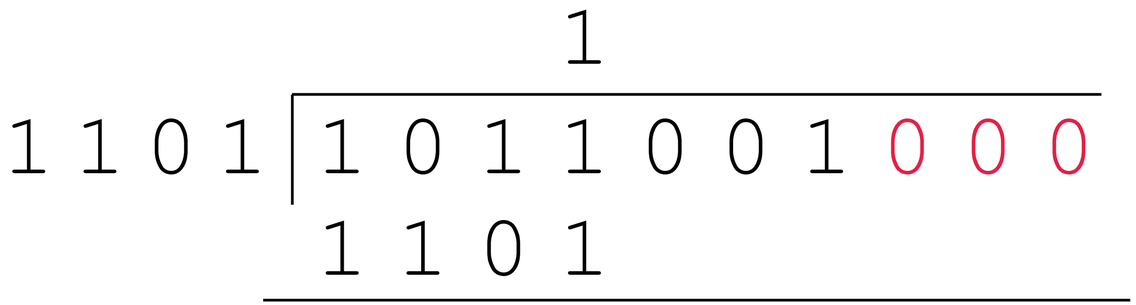
原始数据：1011001

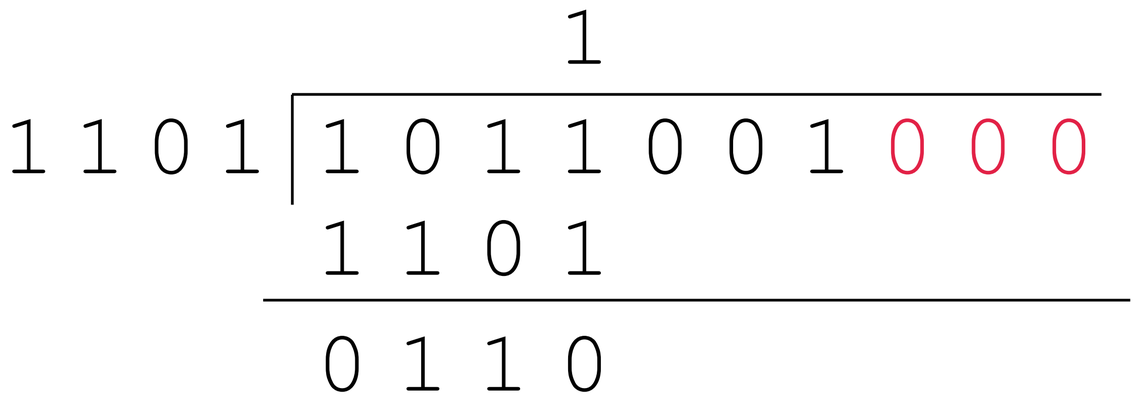
生成多项式：1101

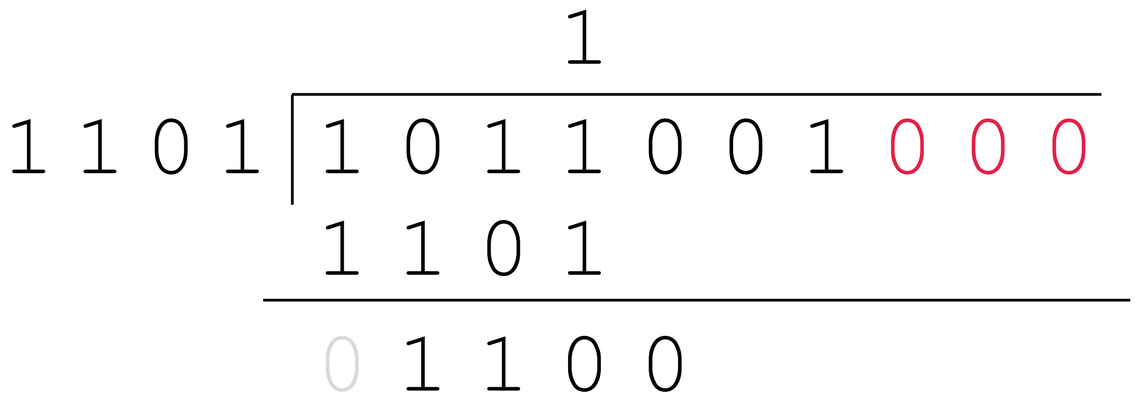



1 1 0 1
1 0 1 1 0 0 1 0 0 0

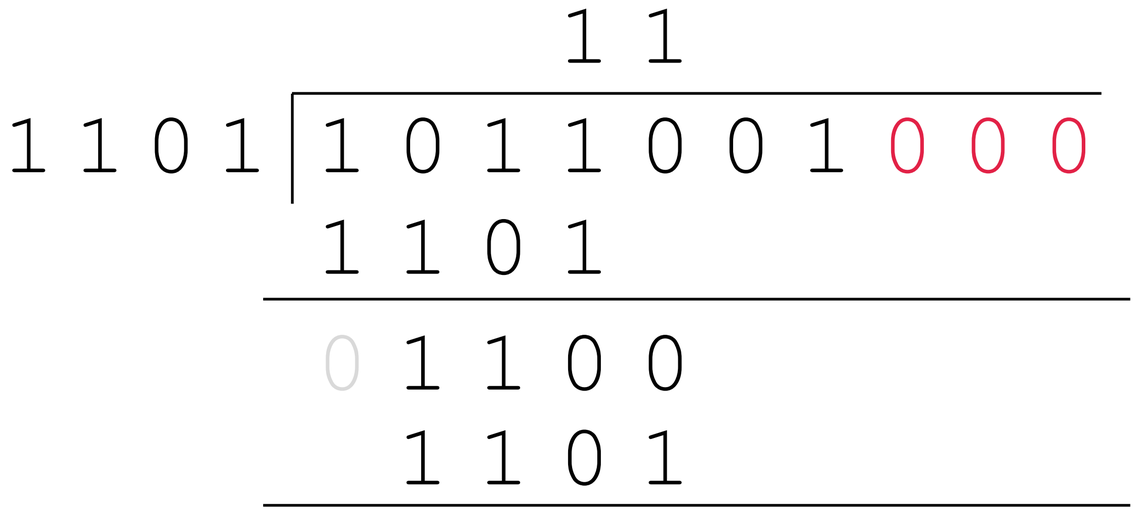

1
1 1 0 1
1 0 1 1 0 0 1 0 0 0

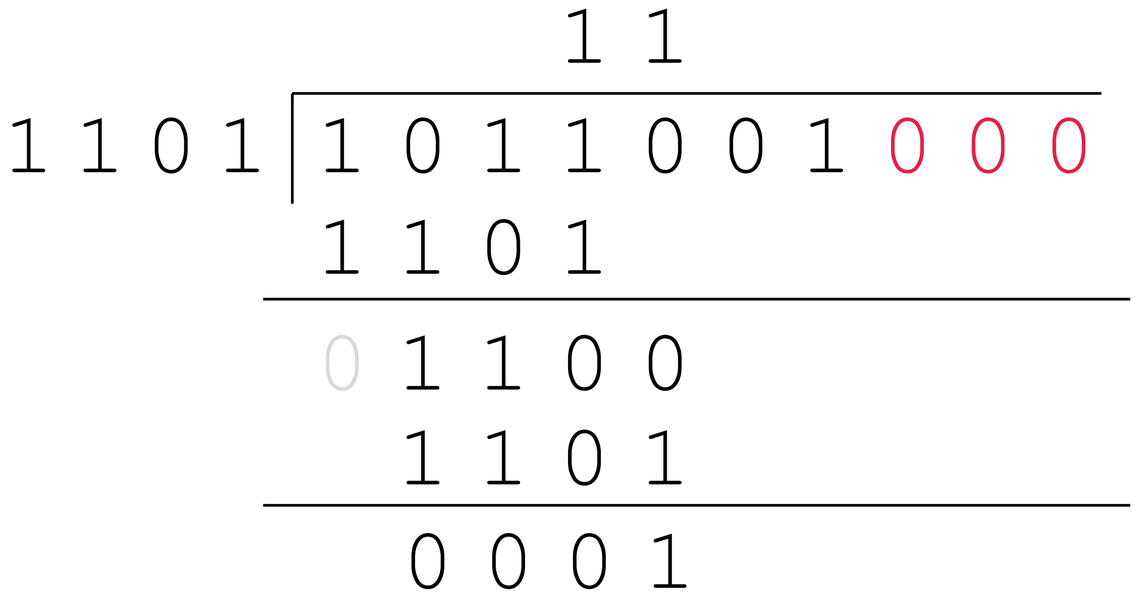

1
1 1 0 1
1 0 1 1 0 0 1 0 0 0
1 1 0 1


1
1 1 0 1
1 0 1 1 0 0 1 0 0 0
1 1 0 1
0 1 1 0

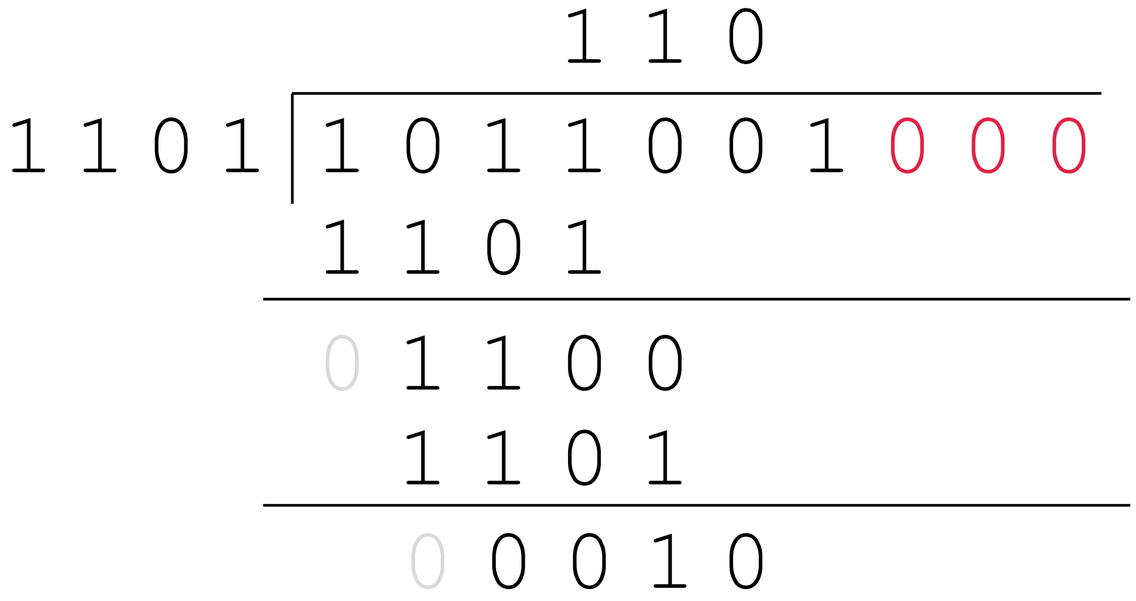

1
1 1 0 1
1 0 1 1 0 0 1 0 0 0
1 1 0 1
0 1 1 0 0

```
          1 1
      ┌──────────────
1101  │1 0 1 1 0 0 1 0 0 0
        1 1 0 1
      ──────────
        0 1 1 0 0
```


1 1
1 1 0 1
1 0 1 1 0 0 1 0 0 0
1 1 0 1
0 1 1 0 0
1 1 0 1


1 1
1 1 0 1
1 0 1 1 0 0 1 0 0 0
1 1 0 1
0 1 1 0 0
1 1 0 1
0 0 0 1

```
          1 1
      ┌──────────────
1101 | 1 0 1 1 0 0 1 0 0 0
       1 1 0 1
     ---------------------
       0 1 1 0 0
         1 1 0 1
     ---------------------
           0 0 0 1 0
```


1 1 0
1 1 0 1 | 1 0 1 1 0 0 1 0 0 0
1 1 0 1
0 1 1 0 0
1 1 0 1
0 0 0 1 0

```
          1 1 0
      ┌─────────────
1101 | 1 0 1 1 0 0 1 0 0 0
       1 1 0 1
     ---------
       0 1 1 0 0
         1 1 0 1
     ---------
         0 0 0 1 0
           0 0 0 0
     ---------
```

```
          1 1 0
      ┌─────────────
1 1 0 1 │ 1 0 1 1 0 0 1 0 0 0
        | 1 1 0 1
        ---------------------
          0 1 1 0 0
            1 1 0 1
        ---------------------
            0 0 0 1 0
              0 0 0 0
        ---------------------
              0 0 1 0
```

```
          1 1 0
      ┌─────────────
1 1 0 1 │ 1 0 1 1 0 0 1 0 0 0
        | 1 1 0 1
        ---------
          0 1 1 0 0
            1 1 0 1
          ---------
            0 0 0 1 0
              0 0 0 0
            ---------
              0 0 1 0 1
```

```
          1 1 0 0
      ┌──────────────
1 1 0 1 │ 1 0 1 1 0 0 1 0 0 0
        | 1 1 0 1
        ---------------------
          0 1 1 0 0
            1 1 0 1
        ---------------------
            0 0 0 1 0
              0 0 0 0
        ---------------------
              0 0 1 0 1
```

```
                1 1 0 0
          ┌─────────────────
1 1 0 1 | 1 0 1 1 0 0 1 0 0 0
          1 1 0 1
        ─────────────────────
          0 1 1 0 0
            1 1 0 1
        ─────────────────────
            0 0 0 1 0
              0 0 0 0
        ─────────────────────
              0 0 1 0 1
                0 0 0 0
        ─────────────────────
```

```
                1 1 0 0
          ┌─────────────────
1 1 0 1 | 1 0 1 1 0 0 1 0 0 0
          1 1 0 1
        ---------------------
          0 1 1 0 0
            1 1 0 1
        ---------------------
            0 0 0 1 0
              0 0 0 0
        ---------------------
              0 0 1 0 1
                0 0 0 0
        ---------------------
                0 1 0 1
```

```
                1 1 0 0
          ┌─────────────────
1 1 0 1 | 1 0 1 1 0 0 1 0 0 0
          1 1 0 1
        ─────────────────────
          0 1 1 0 0
            1 1 0 1
        ─────────────────────
            0 0 0 1 0
              0 0 0 0
        ─────────────────────
              0 0 1 0 1
                0 0 0 0
        ─────────────────────
                0 1 0 1 0
```

```
                  1 1 0 0 1
          ┌──────────────────
1 1 0 1 | 1 0 1 1 0 0 1 0 0 0
          1 1 0 1
        ──────────
          0 1 1 0 0
            1 1 0 1
        ──────────
            0 0 1 0 0
              0 0 0 0
        ──────────
              0 0 1 0 1
                0 0 0 0
        ──────────
                0 1 0 1 0
```

```
                1 1 0 0 1
          ┌─────────────────
1 1 0 1 │ 1 0 1 1 0 0 1 0 0 0
          1 1 0 1
        ─────────
          0 1 1 0 0
            1 1 0 1
        ─────────
            0 0 0 1 0
              0 0 0 0
        ─────────
              0 0 1 0 1
                0 0 0 0
        ─────────
                0 1 0 1 0
                  1 1 0 1
        ─────────
```

```
                  1 1 0 0 1
          ┌──────────────────
1 1 0 1 │ 1 0 1 1 0 0 1 0 0 0
          1 1 0 1
        ──────────
          0 1 1 0 0
            1 1 0 1
        ──────────
            0 0 0 1 0
              0 0 0 0
        ──────────
              0 0 1 0 1
                0 0 0 0
        ──────────
                0 1 0 1 0
                  1 1 0 1
        ──────────
                  0 1 1 1
```

```
                  1 1 0 0 1
          ┌──────────────────
1 1 0 1 | 1 0 1 1 0 0 1 0 0 0
          1 1 0 1
          ---------
            0 1 1 0 0
            1 1 0 1
            ---------
              0 0 0 1 0
                0 0 0 0
                -------
                0 0 1 0 1
                  0 0 0 0
                  -------
                  0 1 0 1 0
                    1 1 0 1
                    -------
                    0 1 1 1 0
```

```
                1 1 0 0 1 1
          ┌─────────────────
1 1 0 1 | 1 0 1 1 0 0 1 0 0 0
          1 1 0 1
        ─────────
          0 1 1 0 0
            1 1 0 1
        ─────────
            0 0 0 1 0
              0 0 0 0
        ─────────
              0 0 1 0 1
                0 0 0 0
        ─────────
                0 1 0 1 0
                  1 1 0 1
        ─────────
                  0 1 1 1 0
```

```
                1 1 0 0 1 1
          ┌─────────────────
1 1 0 1 │ 1 0 1 1 0 0 1 0 0 0
          1 1 0 1
        ─────────
          0 1 1 0 0
            1 1 0 1
        ─────────
            0 0 0 1 0
              0 0 0 0
        ─────────
              0 0 1 0 1
                0 0 0 0
        ─────────
                0 1 0 1 0
                  1 1 0 1
        ─────────
                  0 1 1 1 0
                    1 1 0 1
        ─────────
```

```
                1 1 0 0 1 1
          ┌─────────────────
1 1 0 1 │ 1 0 1 1 0 0 1 0 0 0
          1 1 0 1
        ─────────
          0 1 1 0 0
            1 1 0 1
        ─────────
            0 0 0 1 0
              0 0 0 0
        ─────────
              0 0 1 0 1
                0 0 0 0
        ─────────
                0 1 0 1 0
                  1 1 0 1
        ─────────
                  0 1 1 1 0
                    1 1 0 1
        ─────────
                    0 0 1 1
```

```
                1 1 0 0 1 1
          ┌─────────────────
1 1 0 1 │ 1 0 1 1 0 0 1 0 0 0
          1 1 0 1
        ─────────
          0 1 1 0 0
            1 1 0 1
        ─────────
            0 0 0 1 0
              0 0 0 0
        ─────────
              0 0 1 0 1
                0 0 0 0
        ─────────
                0 1 0 1 0
                  1 1 0 1
        ─────────
                  0 1 1 1 0
                    1 1 0 1
        ─────────
                    0 0 1 1 0
```

```
                1 1 0 0 1 1
          ┌─────────────────
1 1 0 1 │ 1 0 1 1 0 0 1 0 0 0
          1 1 0 1
        ─────────
          0 1 1 0 0
            1 1 0 1
        ─────────
            0 0 0 1 0
              0 0 0 0
        ─────────
              0 0 1 0 1
                0 0 0 0
        ─────────
                0 1 0 1 0
                  1 1 0 1
        ─────────
                  0 1 1 1 0
                    1 1 0 1
        ─────────
                    0 0 1 1 0
```

错误检测 – 循环冗余校验（Cyclic Redundancy Check）

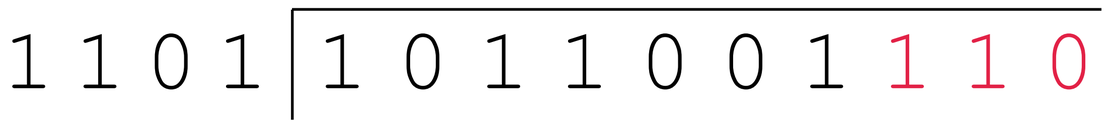
最终发送数据为原始数据 + CRC码：1011001110

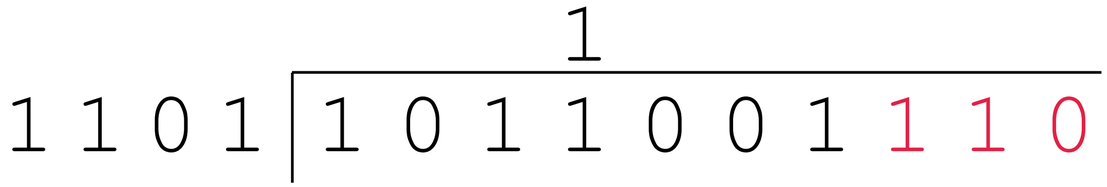
错误检测 – 循环冗余校验（Cyclic Redundancy Check）

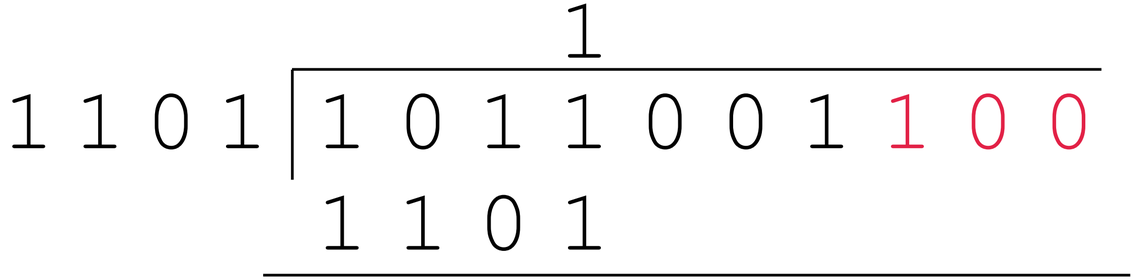
最终发送数据为原始数据 + CRC码：1011001110

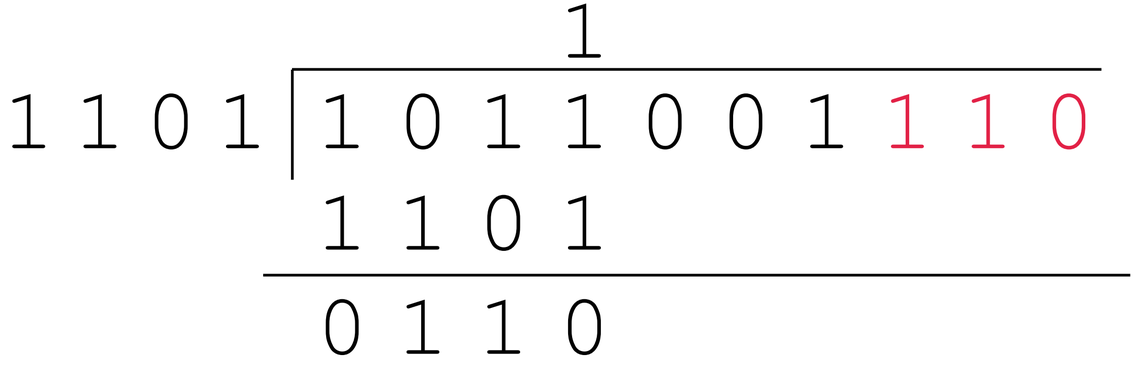
接收方使用生成多项式验证数据是否正确，使用接收到的数据除以生成多项式

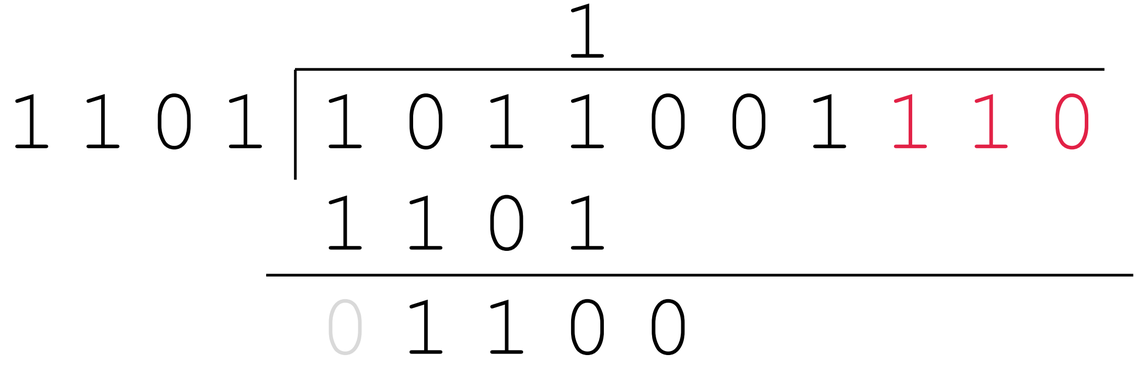
余数为 0 ，说明数据正确无误

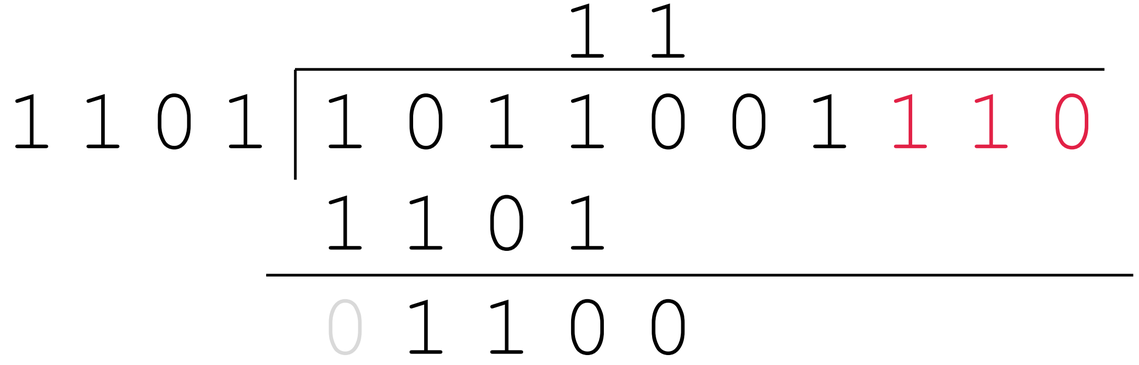

1 1 0 1
1 0 1 1 0 0 1 1 1 0

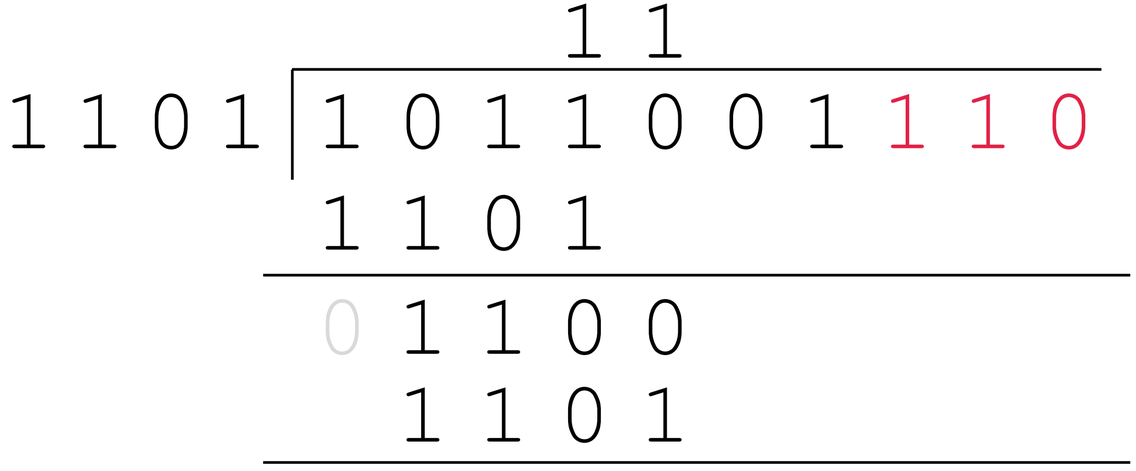

1
1 1 0 1
1 0 1 1 0 0 1 1 1 0

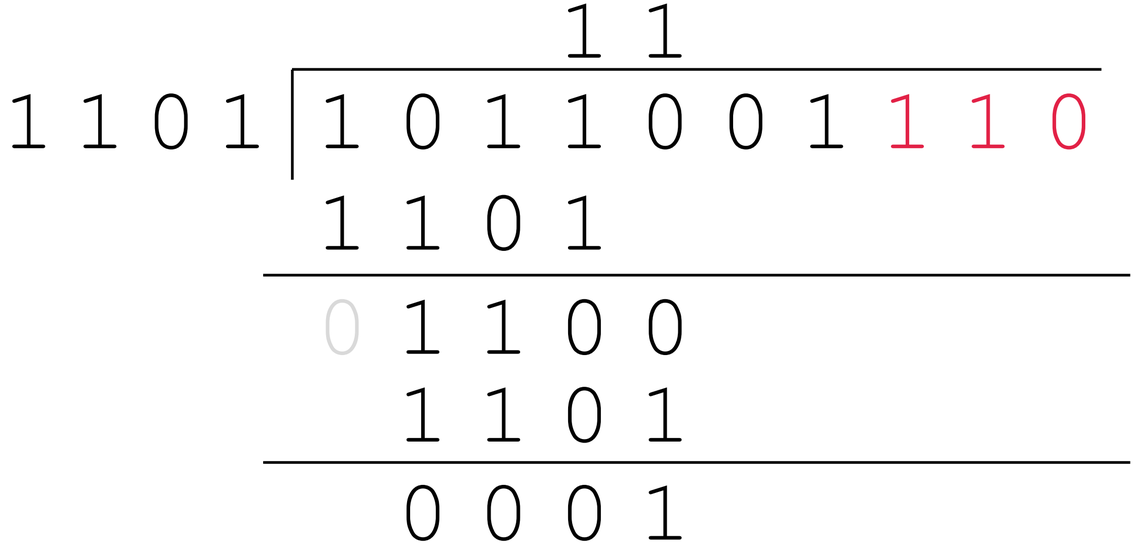

1
1101
1011001100
1101


1
1101
10110011110
1101
0110

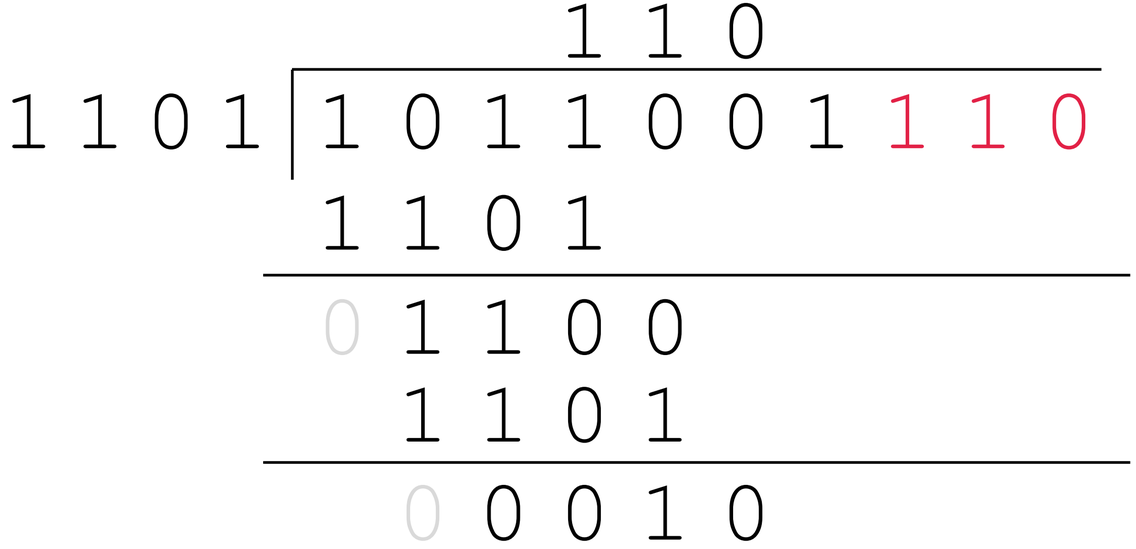

1
1101
10110011110
1101
01100


1 1
1 1 0 1
1 0 1 1 0 0 1 1 1 0
1 1 0 1
0 1 1 0 0


1 1
1 1 0 1
1 0 1 1 0 0 1 1 1 0
1 1 0 1
0 1 1 0 0
1 1 0 1


1 1
1 1 0 1 | 1 0 1 1 0 0 1 1 1 0
1 1 0 1
0 1 1 0 0
1 1 0 1
0 0 0 1

```
          1 1
      ┌─────────────
1 1 0 1 1 0 1 1 0 0 1 1 1 0
        1 1 0 1
      ─────────────
        0 1 1 0 0
          1 1 0 1
      ─────────────
          0 0 0 1 0
```

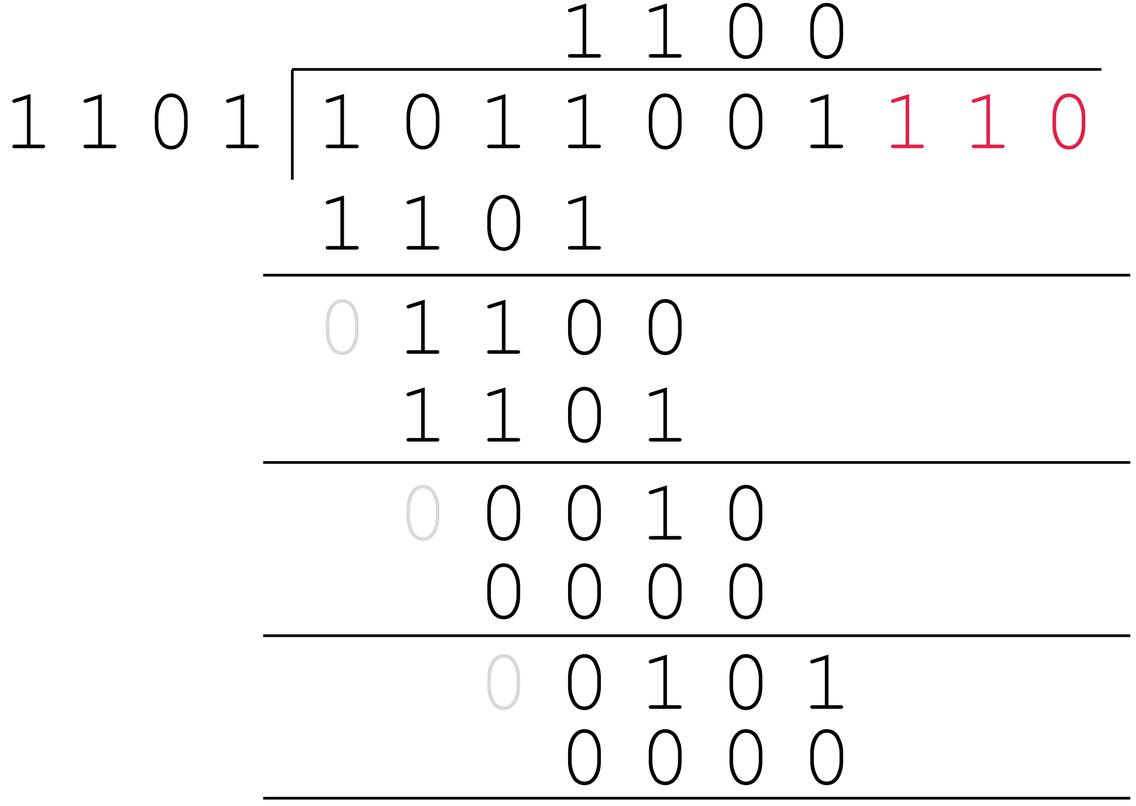

1 1 0
1 1 0 1 | 1 0 1 1 0 0 1 1 1 0
1 1 0 1
0 1 1 0 0
1 1 0 1
0 0 0 1 0

```
          1 1 0
      ┌─────────────
1 1 0 1 │ 1 0 1 1 0 0 1 1 1 0
        │ 1 1 0 1
        ├─────────────────────
        │ 0 1 1 0 0
        │   1 1 0 1
        ├─────────────────────
        │     0 0 1 0
        │       0 0 0 0
        ├─────────────────────
```

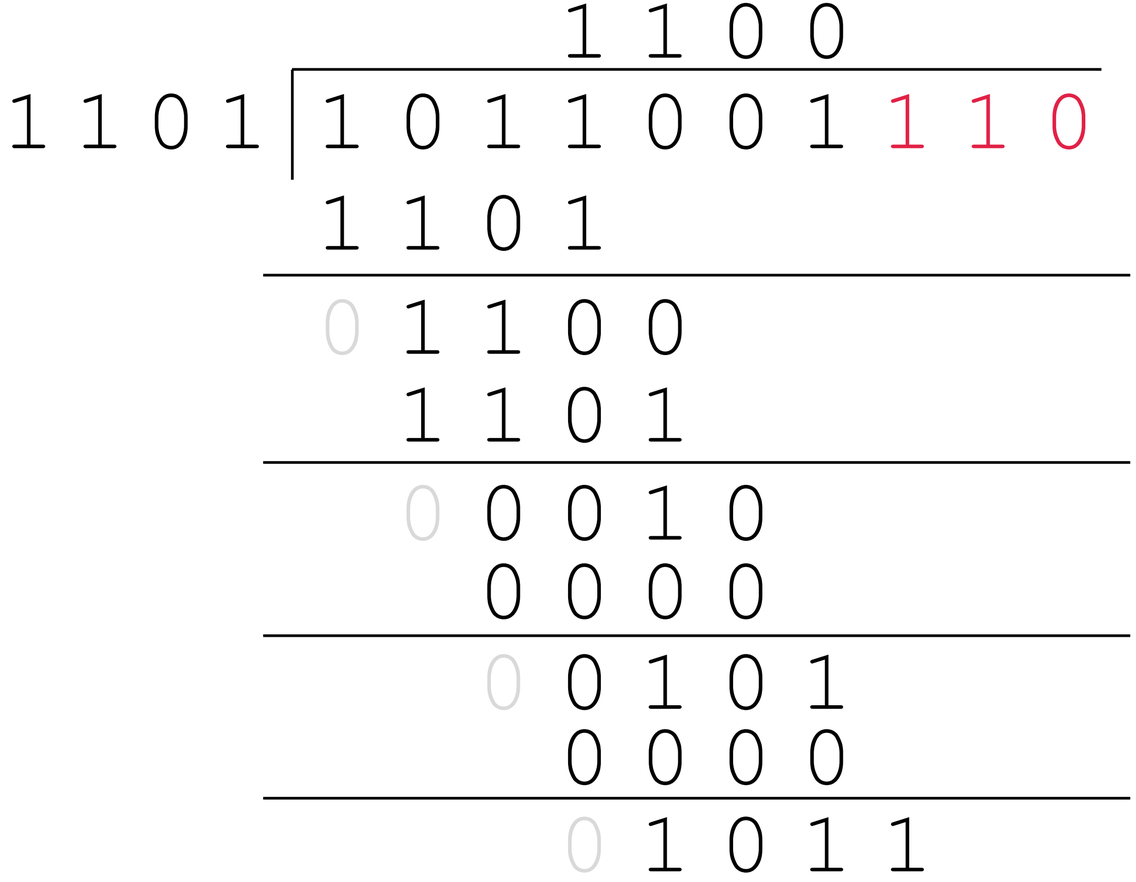
```
                1 1 0
          ┌─────────────
1 1 0 1 │ 1 0 1 1 0 0 1 1 1 0
        │   1 1 0 1
        ├─────────────────────
        │   0 1 1 0 0
        │     1 1 0 1
        ├─────────────────────
        │     0 0 0 1 0
        │       0 0 0 0
        ├─────────────────────
        │       0 0 1 0
```

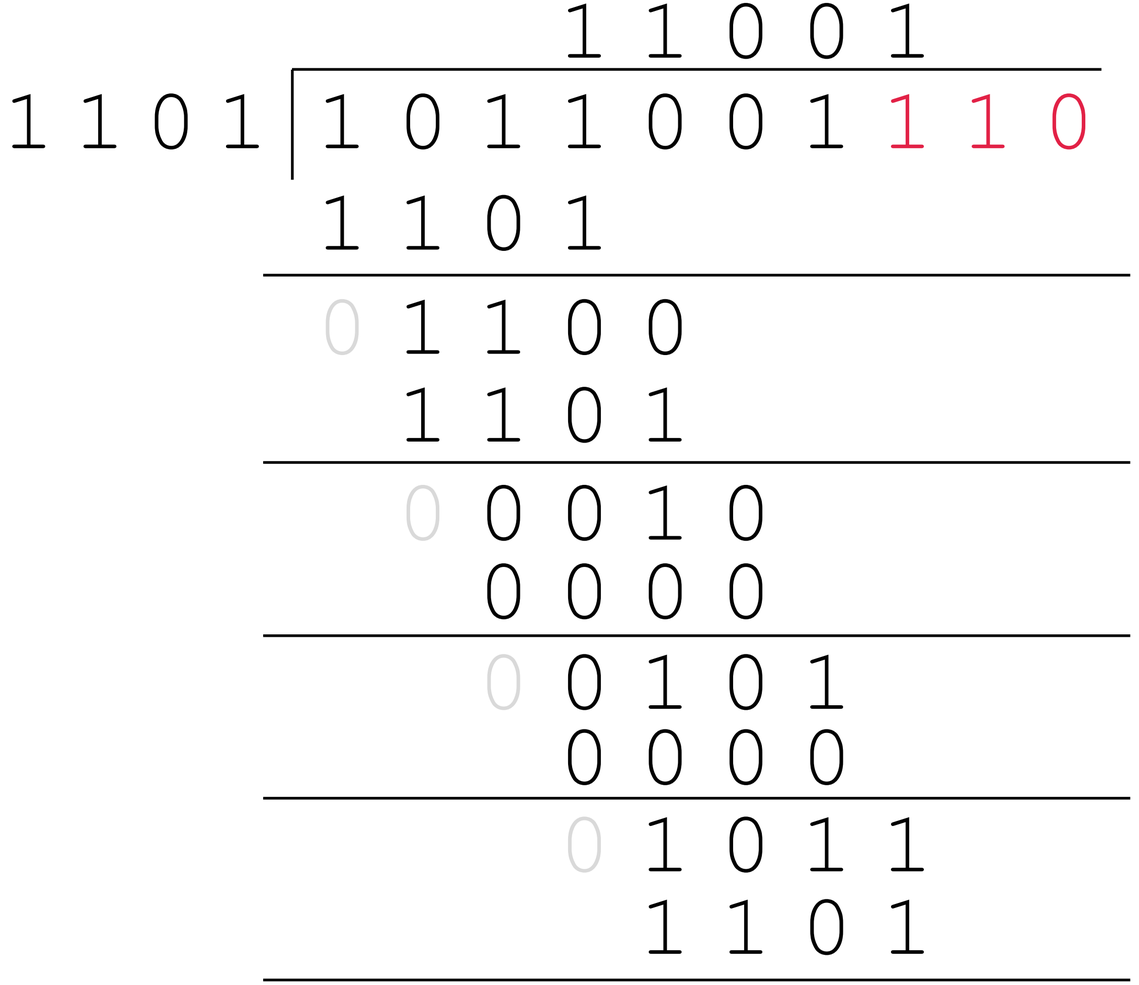
```
          1 1 0
      ┌─────────────
1 1 0 1 │ 1 0 1 1 0 0 1 1 1 0
        │ 1 1 0 1
        ├─────────────────────
        │ 0 1 1 0 0
        │   1 1 0 1
        ├─────────────────────
        │     0 0 1 0
        │       0 0 0 0
        ├─────────────────────
        │       0 0 1 0 1
```

```
          1 1 0 0
      ┌─────────────
1 1 0 1 1 0 1 1 0 0 1 1 0
        1 1 0 1
      ─────────────
        0 1 1 0 0
          1 1 0 1
      ─────────────
          0 0 0 1 0
            0 0 0 0
      ─────────────
            0 0 1 0 1
```


1 1 0 0
1 1 0 1 | 1 0 1 1 0 0 1 1 1 0
1 1 0 1
0 1 1 0 0
1 1 0 1
0 0 0 1 0
0 0 0 0
0 0 1 0 1
0 0 0 0

```
          1 1 0 0
      ┌─────────────
1 1 0 1 1 0 1 1 0 0 1 1 0
        1 1 0 1
      ─────────────
        0 1 1 0 0
          1 1 0 1
      ─────────────
          0 0 0 1 0
            0 0 0 0
      ─────────────
            0 0 1 0 1
              0 0 0 0
      ─────────────
              0 1 0 1
```


1 1 0 0
1 1 0 1 | 1 0 1 1 0 0 1 1 1 0
1 1 0 1
0 1 1 0 0
1 1 0 1
0 0 0 1 0
0 0 0 0
0 0 1 0 1
0 0 0 0
0 1 0 1 1


1 1 0 0 1
1 1 0 1 | 1 0 1 1 0 0 1 1 1 0
1 1 0 1
0 1 1 0 0
1 1 0 1
0 0 0 1 0
0 0 0 0
0 0 1 0 1
0 0 0 0
0 1 0 1 1
1 1 0 1

```
                1 1 0 0 1
      ┌─────────────────────
1 1 0 1 │ 1 0 1 1 0 0 1 1 1 0
        │ 1 1 0 1
        ├─────────────────────
        │   0 1 1 0 0
        │     1 1 0 1
        ├─────────────────────
        │     0 0 0 1 0
        │       0 0 0 0
        ├─────────────────────
        │       0 0 1 0 1
        │         0 0 0 0
        ├─────────────────────
        │           0 1 0 1 1
        │             1 1 0 1
        ├─────────────────────
        │             0 1 1 0
```

```
                1 1 0 0 1
      ┌─────────────────────
1 1 0 1 | 1 0 1 1 0 0 1 1 1 0
          1 1 0 1
        ---------
          0 1 1 0 0
            1 1 0 1
        ---------
            0 0 0 1 0
              0 0 0 0
        ---------
              0 0 1 0 1
                0 0 0 0
        ---------
                0 1 0 1 1
                  1 1 0 1
        ---------
                    0 1 1 0 1
```

```
                1 1 0 0 1 1
        ┌──────────────────
1 1 0 1 │ 1 0 1 1 0 0 1 1 1 0
          1 1 0 1
        ──────────
          0 1 1 0 0
            1 1 0 1
        ──────────
            0 0 0 1 0
              0 0 0 0
        ──────────
              0 0 1 0 1
                0 0 0 0
        ──────────
                0 1 0 1 1
                  1 1 0 1
        ──────────
                    0 1 1 0 1
```

```
                1 1 0 0 1 1
          ┌─────────────────
1 1 0 1 | 1 0 1 1 0 0 1 1 1 0
          1 1 0 1
        ─────────────────────
          0 1 1 0 0
            1 1 0 1
        ─────────────────────
            0 0 0 1 0
              0 0 0 0
        ─────────────────────
              0 0 1 0 1
                0 0 0 0
        ─────────────────────
                0 1 0 1 1
                  1 1 0 1
        ─────────────────────
                  0 1 1 0 1
                    1 1 0 1
        ─────────────────────
                      0 0 0 0
```

```
                1 1 0 0 1 1
      ┌─────────────────────
1 1 0 1 │ 1 0 1 1 0 0 1 1 1 0
        │ 1 1 0 1
        ├─────────────────────
        │   0 1 1 0 0
        │     1 1 0 1
        ├─────────────────────
        │     0 0 0 1 0
        │       0 0 0 0
        ├─────────────────────
        │       0 0 1 0 1
        │         0 0 0 0
        ├─────────────────────
        │         0 1 0 1 1
        │           1 1 0 1
        ├─────────────────────
        │           0 1 1 0 1
        │             1 1 0 1
        ├─────────────────────
        │             0 0 0 0 0
```

```
                1 1 0 0 1 1 0
          ┌─────────────────────
1 1 0 1 │ 1 0 1 1 0 0 1 1 1 0
          1 1 0 1
        ─────────────────────
          0 1 1 0 0
            1 1 0 1
        ─────────────────────
            0 0 0 1 0
              0 0 0 0
        ─────────────────────
              0 0 1 0 1
                0 0 0 0
        ─────────────────────
                0 1 0 1 1
                  1 1 0 1
        ─────────────────────
                  0 1 1 0 1
                    1 1 0 1
        ─────────────────────
                    0 0 0 0 0
                      0 0 0 0
        ─────────────────────
```

```
                1 1 0 0 1 1 0
          ┌─────────────────────
1 1 0 1 │ 1 0 1 1 0 0 1 1 1 0
          1 1 0 1
        ─────────────────────
          0 1 1 0 0
            1 1 0 1
        ─────────────────────
            0 0 0 1 0
              0 0 0 0
        ─────────────────────
              0 0 1 0 1
                0 0 0 0
        ─────────────────────
                0 1 0 1 1
                  1 1 0 1
        ─────────────────────
                  0 1 1 0 1
                    1 1 0 1
        ─────────────────────
                    0 0 0 0 0
                      0 0 0 0
        ─────────────────────
                        0 0 0
```

为什么1101叫做“生成多项式”？

CRC中的多项式实际上是“二进制多项式”

- 多项式的系数只可能是0或1，而不可能是其他数字
- 系数为1的项表示存在该项，系数为0表示不存在该项

为什么1101叫做“生成多项式”？

CRC中的多项式实际上是“二进制多项式”

- 多项式的系数只可能是0或1，而不可能是其他数字
- 系数为1的项表示存在该项，系数为0表示不存在该项

$$x^3 + x + 1 \longrightarrow \mathbf{1} * x^3 + \mathbf{0} * x^2 + \mathbf{1} * x^1 + \mathbf{1} * x^0 \longrightarrow 1011$$

$$x^4 + x^3 + x + 1 \longrightarrow 11011$$

生成多项式是如何确定的?

生成多项式不是随机的，也不是根据数据临时决定的，是事先由协议规定好的，固定不变的

- 每一种协议在制定之初就明确规定了使用的CRC生成多项式
- 不同协议使用的生成多项式不同，且标准化
- 发送端和接收端都按照协议规定使用相同的固定生成多项式进行计算

数据链路层常用错误检测方法对比

检测方法	附加开销	实现难度	错误检测能力	常用协议
奇偶校验	1 bit	简单	弱（偶数位错漏检）	串行通信
检验和	16 bit	较简单	中等	TCP / UDP
CRC 校验	16 或 32 bit	中等	非常强	Ethernet, Wi-Fi, HDLC